

Code Step Guide: Using Previous Step Outputs as Inputs

The **Code** step lets you write custom TypeScript logic inside a workflow. Its superpower is the ability to receive data from any previous step — like output from **Automate Data Board** — and transform or process it with full code control.

How It Works

The Code step has two parts:

INPUTS (key → value from previous step)

```
recordId → {{step_1.boardResponse._id}}  
email    → {{step_1.boardResponse.fields.abc123}}
```

↓ passed as `inputs` object

CODE (TypeScript)

```
export const code = async (inputs) => {  
  return inputs.recordId;  
}
```

1. **Inputs** — a key/value dictionary where each value is a reference to a previous step's output (selected via the variable picker `{{...}}`).
2. **Code** — TypeScript code that receives those values as the **inputs** object and returns a result.

Step-by-Step Usage

1. Add a Code Step After a Data Board Step

Place a **Code** step after any step that produces output, for example after an **Automate Data Board** (`search`, `getAll`, `create`, etc.).

2. Map Previous Outputs to Inputs

In the **Inputs** section, add key/value pairs:

Key (your variable name)	Value (pick from previous step)
recordId	{{step_1.boardResponse._id}}
fields	{{step_1.boardResponse.fields}}
records	{{step_1.data}}

- Click the value field and use the **variable picker** `{{}}` to browse available outputs from each previous step.
- Use any name for the key — this becomes `inputs.<key>` in your code.

3. Write Your Code

The entry point **must** be `export const code`. Renaming or removing it will cause the step to fail.

```
export const code = async (inputs: {
  recordId: string;
  fields: Record<string, any>;
}) => {
  // Access previous step data via inputs
  const id = inputs.recordId;
  const name = inputs.fields['Name'];

  // Your logic here
  return {
    processed: true,
    id,
    name,
  };
};
```

The value returned from `code` becomes this step's output, which can be referenced by later steps.

Working with Data Board Outputs

Below are common patterns using outputs from the **Automate Data Board** connector.

Pattern 1: Process a single record (search or create/update output)

Data Board output shape:

```
{
  "boardResponse": {
```

```

    "_id": "rec_abc123",
    "public_id": "PUB-001",
    "fields": {
      "6507a1b2c3d4e5f6": "John Doe",
      "6507a1b2c3d4e5f7": "john@example.com"
    }
  }
}

```

Inputs mapping: | Key | Value | |---| | record | {{step_1.boardResponse}}

Code:

```

export const code = async (inputs: { record: any }) => {
  const { _id, public_id, fields } = inputs.record;

  return {
    id: _id,
    publicId: public_id,
    name: fields['6507a1b2c3d4e5f6'],
    email: fields['6507a1b2c3d4e5f7'],
  };
};

```

Pattern 2: Process a list of records (getAll with outputData = "listRecords", version = "2")

Data Board output shape (version 2, listRecords):

```

{
  "data": [
    {
      "Name": { "value": "John Doe", "type": "ShortText" },
      "Email": { "value": "john@example.com", "type": "Email" },
      "Status": { "value": "opt_active", "type": "SingleSelection" },
      "_id": "rec_1",
      "public_id": "PUB-001"
    }
  ]
}

```

Inputs mapping: | Key | Value | |---| | records | {{step_1.data}}

Code:

```

export const code = async (inputs: { records: any[] }) => {
  const results = inputs.records.map((record) => ({

```

```

    id: record._id,
    name: record['Name']?.value,
    email: record['Email']?.value,
    status: record['Status']?.value,
  }));

  return { count: results.length, results };
};

```

Pattern 3: Filter and transform records

```

export const code = async (inputs: { records: any[] }) => {
  const active = inputs.records.filter(
    (r) => r['Status']?.value === 'opt_active',
  );

  const emails = active.map((r) => r['Email']?.value).filter(Boolean);

  return { activeCount: active.length, emails };
};

```

Pattern 4: Build a contentJson payload for the next Data Board step

When the next step is a Data Board **create** or **update** with `inputMethod = "json"`, you can build the `contentJson` in a Code step:

```

export const code = async (inputs: {
  name: string;
  email: string;
  phone: string;
}) => {
  // Return a clean JSON object ready to pass as contentJson
  return {
    Name: inputs.name,
    Email: inputs.email,
    Phone: inputs.phone,
    Status: 'opt_active',
  };
};

```

Then in the next Data Board step, set `contentJson` to `{{code_step.output}}`.

Accessing Nested Fields

Use dot notation in the variable picker or in code:

Data Board Output	Variable Picker Reference	inputs Access in Code
Record ID	<code>{{step_1.boardResponse.inputs.recordId}}</code>	<code>inputs.recordId</code>
All fields object	<code>{{step_1.boardResponse.inputs}}fields</code>	<code>inputs.fields</code>
Specific field by ID	<code>{{step_1.boardResponse.inputs.fields.fieldID}}</code>	<code>inputs.fields[fieldID]</code>
List of records (v2)	<code>{{step_1.data}}</code>	<code>inputs.records</code>
First record in list	<code>{{step_1.data[0]}}</code>	<code>inputs.firstRecord</code>

Adding npm Packages

If your workflow environment allows npm packages (enabled by the **Allow NPM Packages** feature flag), click **Add package** in the editor toolbar and search for any package. It will be added to the **Dependencies** tab (package.json).

```
import Dayjs from 'dayjs';

export const code = async (inputs: { dateStr: string }) => {
  const formatted = Dayjs(inputs.dateStr).format('DD MMM YYYY');
  return { formatted };
};
```

Important Rules

1. **export const code is required.** Do not rename or remove it.
2. **code must be an async function** — it can await async operations.
3. **Inputs are typed at runtime** — use TypeScript type annotations for safety, but the actual values come from the workflow at execution time.
4. **Return value becomes this step's output** — any serializable object or primitive is valid.
5. **Inputs are injected, not imported** — you don't import previous step data; it arrives via the `inputs` parameter.

Example: Full Flow Using Data Board → Code Step

[Trigger] New form submission
↓

```

[Step 1] Automate Data Board - search by email (fieldsOnBoard)
  ↓ outputs: boardResponse[0]._id, boardResponse[0].fields
[Step 2] Code - decide what to do next
  inputs:
    existingRecord → {{step_1.boardResponse[0]}}
    formEmail      → {{trigger.email}}
  code:
    export const code = async (inputs) => {
      if (!inputs.existingRecord) {
        return { action: 'create', email: inputs.formEmail };
      }
      return {
        action: 'update',
        recordId: inputs.existingRecord._id,
        email: inputs.formEmail,
      };
    };
  ↓ outputs: { action: "update", recordId: "rec_abc123", email: "..." }
[Step 3] Automate Data Board - update record using {{step_2.output.recordId}}

```